

Carta de fechamento

DOC Intelligence — Trilha B (front-end)

Leopoldo Couto · 1º de setembro de 2026

Repositório: <https://github.com/eng-leopoldocouto/DOC-Intelligence>

O que ficou de fora, e por quê

A **busca por termo** ficou de fora inteira — nem contrato, nem tela. O que existe é listagem filtrada por estado, com cursor, que é o que a fila consome. A fatia que o enunciado nomeia vai "do envio até a correção de um campo"; busca é uma segunda fatia, e construí-la pela metade produziria exatamente o que o enunciado penaliza. Ficaram também de fora o upload retomável, a priorização da fila, a deduplicação perceptual e a trilha de auditoria de leitura. Cada uma está em 07-nao-feito.md com custo estimado e o **gatilho** que a tornaria necessária, porque "não fiz" sem estimativa é opinião.

Vale dizer como essa frase chegou a esta forma. Até a última rodada, quatro documentos meus diziam que a busca estava "projetada e servida pelo mock" — e o `openapi.yaml` não tem parâmetro de termo nenhum. Podia tê-la tornado verdadeira acrescentando o parâmetro em vinte minutos, e **escolhi corrigir o texto**: acrescentar superfície de contrato sem consumidor, no fim do prazo, para justificar uma frase escrita antes, é a mesma falha com o sinal trocado.

Duas ausências me incomodam mais que as outras. O **HEIC do iPhone** é recusado com uma mensagem que ensina a mudar o ajuste da câmera: é solução organizacional para problema técnico, funciona, e depende de treinamento — que se perde na rotatividade. E a **priorização da fila** ficou de fora porque depende de uma pergunta que não tive tempo hábil de enviar ao senhor: existe SLA de mesmo dia? Assumi que não, documentei a premissa, e ela é a primeira coisa a mudar se eu estiver errado.

O que quebra primeiro se o volume for multiplicado por dez

Não é a interface — é a fila de conferência, e nenhuma decisão de front-end resolve isso.

A 1.500 documentos por dia e 8.000 no pico, os gargalos técnicos que tratei seguem de pé: o polling continua sendo uma requisição para N documentos, a fila continua paginada por cursor, o upload continua limitado a três simultâneos. O que quebra é aritmética. Duas pessoas do atendimento, aos quatro minutos por documento que o enunciado cita, conferem cerca de 240 por dia. Com 8.000 chegando entre 9h e 11h e uma fração relevante caindo na conferência por baixa confiança, **a fila não drena — ela acumula indefinidamente.**

Tratei o que a interface pode tratar, e não mais que isso: o cabeçalho da fila agora mostra quantos aguardam e há quanto tempo espera o mais antigo, com destaque acima de um limite derivado dos números do próprio enunciado — quatro minutos por documento, duas pessoas, cerca de trinta por hora. **Isso não faz a fila drenar.** Faz o problema parar de ser invisível para quem decide, que é coisa diferente e é o teto do que um front-end alcança aqui.

O resto é aritmética de pessoal, e é decisão de gestão: elevar o limiar de confiança que dispensa conferência, aceitar amostragem em vez de conferência integral, ou contratar. Nenhuma delas é minha para tomar — mas é minha a obrigação de mostrar a conta, e antes desta rodada eu não estava mostrando.

Qual das minhas decisões eu menos defenderia hoje

O PATCH que grava e **conclui** a conferência na mesma operação, sem permitir salvar rascunho.

Meu argumento foi legítimo: rascunho num documento reservado por TTL cria um estado que ninguém sabe interpretar quando a reserva expira, e preferi não ter a funcionalidade a ter uma que confunde. Mas o custo real cai sobre quem usa. A pessoa que está conferindo um contracheque com seis campos, é chamada no balcão e volta dez minutos depois, **perde tudo**. Otimizei para a limpeza do meu modelo de estados em vez de otimizar para o operador — e num sistema que existe para poupar quatro minutos por documento, fazer alguém repetir dez é o tipo de troca que eu não faria de novo.

A correção não é difícil: rascunho local que expira junto com a reserva, com aviso explícito de que não está salvo no servidor. Não a fiz por prazo, e é a primeira coisa que eu mudaria.

A resposta não mudou na última rodada — ficou pior. Ao escrever a fronteira de erro que faltava, tive de redigir o que a tela diz quando quebra no meio de uma conferência: *"se você acabou de corrigir campos sem salvar, o que estava na tela se perdeu"*. Um defeito que eu justificava pela limpeza do modelo de estados passou a ter uma tela dedicada a informar a perda.

Quanto tempo isso tudo levou

Pelo menos 5h34 de trabalho efetivo, somando a coluna de duração de docs/ia/registro-de-tempo.md: 3h20 na noite de 31 de agosto, mais 1h35 já fechadas na manhã seguinte, mais a fase de fechamento em curso. Houve uma interrupção de cerca de duas horas por limite de uso da ferramenta, não contabilizada.

O "pelo menos" não é modéstia — é a correção de um erro que este arquivo cometeu quatro vezes. Nas três primeiras eu escrevi números que ainda não tinham acontecido. Na quarta, apontada pela última auditoria, o erro foi o oposto e mais difícil de ver: **o registro parava na madrugada** e declarava 3h20, enquanto o `git log` já mostrava sete commits da manhã seguinte, duas ADRs novas, CI, hook, linter e a suíte indo de 64 para 96 testes. A soma da coluna estava certa; o que faltava era coluna. $\geq$ é verdade em qualquer instante posterior; um número exato só seria verdade se eu acertasse o futuro.

Devo uma ressalva sobre esse arquivo, e ela é a coisa mais útil que aprendi nesta entrega. Ele trazia horários **estimados para a frente** — o commit que os gravou é anterior ao horário declarado como início. O auditor apontou. Corrigi, **e reincidi na correção duas vezes**: primeiro escrevendo outro carimbo futuro dentro do parágrafo que acabara de explicar por que isso é inaceitável; depois mudando dois valores da coluna e repetindo o total antigo no mesmo diff.

O conserto definitivo não foi um número melhor: foi tirar do arquivo a possibilidade de errar — a coluna "fim" da fase em andamento passou a dizer *"ver último commit"*. E a terceira auditoria nomeou o padrão que eu não via: *o defeito não está no item apontado — está no vizinho aritmético dele, ou no outro documento que o cita*. Daí a regra que passei a seguir: **procurar pelo número, não pelo assunto**. Registre as três quedas em vez de apagá-las; elas dizem mais sobre como eu trabalho do que a versão limpa diria.

A distribuição diz mais que o total: **por volta de 40% do tempo foi spec, ADRs e registro de decisão, antes de existir uma linha de código**. A especificação inteira e as **treze primeiras** ADRs saíram em vinte e dois minutos de escrita; o plano de implementação, em dezoito. Foi o melhor investimento da entrega — a fatia vertical foi construída sem uma única decisão de arquitetura tomada no meio do código, e as **onze** divergências que apareceram estão registradas contra uma spec congelada numa tag desde antes do primeiro commit de implementação. Quatro delas só existem porque um subagente auditor, em contexto frio, encontrou o que eu não tinha encontrado no meu próprio texto; duas, porque uma auditoria externa encontrou o que nem eu nem ele achamos. Duas estão **fechadas**: o linter e a CI, que passou a provar por comando o que o README afirmava por escrito.

Trabalhei com o Claude Opus 5 em sessão interativa. O prazo foi comprimido de três dias para menos de um por decisão minha, o que tornou o recorte mais importante que a execução — e é por isso que a maior parte do que entrego é texto, e não código.

Se houver uma única coisa a levar desta entrega sobre trabalhar com agentes, é esta: **o autor não consegue auditar o próprio texto contra o próprio código**, porque lê o texto e lembra da intenção em vez de ver o que ficou. O valor do subagente auditor não foi capacidade técnica — foi não ter memória do que eu quis dizer. Rodei-o **quatro vezes**, e as quatro encontraram algo: a segunda achou o que a primeira correção tinha introduzido, e a terceira, o que a segunda introduziu. O agente não erra por incompetência; erra por escopo de atenção — corrige com precisão o que foi apontado e não olha ao lado. **Eu também não. A diferença é que eu achava que estava olhando.**

A última rodada acrescentou a segunda metade da lição. Restaram quatro erros depois de tudo pronto, e **nenhum foi pego por teste** — passaram por tipo estrito, por linter, por noventa e seis testes e pela minha leitura. Dois morreram em menos de um minuto de navegador aberto; um, refazendo uma medição que eu tinha aceitado; o último, relendo código enquanto esperava o auditor. **Teste verifica o que alguém pensou em verificar. Abrir a tela mostra o que ninguém pensou — e reler o que já passou mostra o resto.**